```
function printSong(): void {
    include __DIR__ . "/include_scope1.php";
};

printSong();
```

```
Output:
Warning: Undefined variable $song in /usr/src/myapp/include_scope1.php
on line 3
```

This time we're including `include_scope1.php` during a function body. `$song` is still a global variable, so it isn't accessible from the scope of `printSong()` with predictable results. If we need this code to work, there is a mechanism for making out-of-scope variables available to a function, which we'll see in action when we look at anonymous functions later.

New Function, New (Local) Scope

So far we've looked at passing control to and from single functions, jumping between local and global scope. It's worth making the point that local scope is specific to each function, here's a quick demonstration:

```
function outer(): void
{
    $funcName = 'outer()';
    inner();
    print $funcName . PHP_EOL;
}

function inner(): void
{
    $funcName = 'inner()';
    print $funcName . PHP_EOL;
}

outer();
```